

## Q1.

實作上採用了 Hugging Face 生態系統進行模型微調，並利用 4-bit 量化技術降低硬體需求 (QLoRA)。主要使用的 Python 套件包含：

- **Transformers:** 負責載入預訓練語言模型 (AutoModelForSequenceClassification)、分詞器 (AutoTokenizer) 以及訓練流程控制 (Trainer)。
- **PEFT:** 負責實作 LoRA 微調，透過 LoraConfig 設定低秩適應的目標模組。
- **BitsAndBytes:** 處理模型的 4-bit 量化 (bnb\_4bit\_quant\_type="nf4") 與分頁優化器 (paged\_adamw\_8bit)。
- **PyTorch:** 作為底層的深度學習運算框架，處理張量運算與自定義損失函數。
- **Sentence-Transformers:** 用於將使用者的提問轉換為向量，以執行後續的 KNN 殘差修正。
- **NumPy / Pandas:** 負責資料集的預處理、預測結果的陣列運算與 CSV 輸出。

路由器的骨幹網路採用了 Qwen/Qwen3-4B-Base 與 Qwen/Qwen3-8B-Base 語言模型，並透過添加不同的分類或回歸頭來實現路由決策。程式碼中包含了兩種訓練途徑：

### 1. Independent CARROT Router

- 訓練兩個獨立的模組：一個預測候選模型在該提問下的**效能**，另一個預測**成本**。
- 效能預測被視為多標籤分類問題，而成本預測則為回歸問題。
- 最終透過公式計算綜合預期分數： $0.85 * \text{預測效能} - 0.15 * \text{正規化成本}$ ，選擇分數最高的模型。

### 2. Soft-Oracle Classifier

- 將計算好的真實 Reward 轉換為機率分佈。
- 直接訓練單一分類器，預測該提問導向各個候選模型的最佳機率。

針對不同的架構，程式碼設計了特製的損失函數以優化路由決策：

- **Routing-Aware Loss (用於效能預測)：**結合了二元交叉熵與排序損失。
  - **BCE Loss：**計算預測效能與真實效能的誤差，並乘上基於 Reward 差距計算出的 row\_weights，給予關鍵決策點更高的權重。
  - **Ranking Loss：**計算候選模型間的分數差距，透過 Softplus 函數優化排序，確保模型能正確分辨最好與次好的候選者。
- **KL Divergence Loss (用於 Soft-Oracle)：**
  - 計算預測輸出與真實 Reward 轉換之機率分佈之間的 KL 散度，同樣搭配 row\_weights 進行樣本加權。

為了解決模型在驗證集上的偏差，程式碼實作了兩道防線：

- **Coordinate Search Bias Calibration (calibrate\_soft\_oracle\_bias.py)：**
  - 針對驗證集輸出的 Logits 進行 Coarse-to-fine 的座標軸搜尋，尋找能最大化 Validation Reward 的 Logit 偏差值。
  - 為了防止過度擬合驗證集，最終找出的 Bias 會乘上一個縮放因子 (0.7)。

- **KNN Residual Correction :**

- 利用 `intfloat/e5-base-v2` 模型將測試集提問轉為向量。
- 在訓練集中尋找最相似的 50 個鄰居，計算這些鄰居的真實分數與預測分數的殘差。
- 將殘差進行加權平均後，作為最終預測分數的修正項。

以下為統整程式碼中預設的關鍵超參數：

| 參數類別                | 參數名稱                    | 設定值 / 策略                        |
|---------------------|-------------------------|---------------------------------|
| <b>LoRA 微調</b>      | Rank (r), Alpha         | r=16, lora_alpha=32             |
|                     | Dropout                 | 0.05                            |
|                     | Target Modules          | q, k, v, o, gate, up, down_proj |
| <b>訓練優化</b>         | Learning Rate           | 3e-5 (Linear Scheduler)         |
|                     | Optimizer               | paged_adamw_8bit                |
|                     | Warmup Ratio            | 0.1                             |
|                     | Weight Decay            | 0.01                            |
|                     | Batch Size / Max Length | batch_size=1, max_length=512    |
| <b>Ranking Loss</b> | Loss Weight             | 1.0                             |
|                     | Temperature             | 0.05                            |
|                     | Min Gap                 | 1e-4                            |

|         |                             |                                                |
|---------|-----------------------------|------------------------------------------------|
| Bias 校正 | 搜尋網格 (Step, Radius, Passes) | (0.10, 1.5, 3), (0.05, 0.5, 3), (0.02, 0.2, 2) |
|         | Shrink Factor               | 0.7                                            |

## Q2.

### 1. 效能與成本的平衡機制

在平衡效能與成本上，設計的核心邏輯是**對齊 Kaggle 競賽的評估指標**。競賽採用

$$Reward_{0.85} = 0.85 * \bar{P} - 0.15 \times \frac{\bar{C}}{C_{max}}$$

作為計分標準。

實作中，我們不只讓模型預測效能，還將成本懲罰直接寫入訓練目標與推論邏輯中：

- **成本正規化**：為了讓成本能與介於 0 到 1 之間的效能指標抗衡，在 `carrot_router.py` 中，我們透過計算 `global_max_cost` 作為分母，將所有模型的絕對成本轉換為相對的正規化成本 (`normalized_cost`)。
- **動態計算分數**：在進行路由決策時，最終選擇的依據是綜合預期分數 (`0.85 * predicted_performance - (0.15 * normalized_cost)`)。透過給定 0.85 的效能權重與 0.15 的成本權重，強迫模型在「選擇昂貴的高效能模型」與「選擇便宜的堪用模型」之間做出數學上的最佳妥協。

### 2. 路由器設計動機

我們嘗試了兩種不同的架構 (Independent CARROT 與 Soft-Oracle)，並搭配了事後校正機制，其背後的设计動機如下：

#### A. 雙軌制的架構探索

- **Independent CARROT (解耦預測)**：效能與成本的潛在特徵可能完全不同 (例如：某個提問需要複雜推理→效能差異大，但因為其字數短→成本差異小)。因此，我們設計了雙頭架構，將效能視為多標籤分類問題，成本視為迴歸問題。讓模型先分別理解模型能力與運算代價，最後再由數學公式進行統合。
- **Soft-Oracle Classifier (端到端分佈擬合)**：有時候解耦預測會累積誤差。因此我們設計了另一套做法，直接算出 11 個候選模型的真實 Reward，並加上溫度參數 (`soft_label_temperature`) 轉換為 Softmax 機率分佈。透過直接教導分類器學習最佳的取捨分佈，讓模型學會效能與成本的平衡點。

## B. 基於「遺憾值」的感知學習

在真實情境中，選錯的代價並非等價的。

- **設計動機**：如果針對某個提問，Model A 的 Reward 是 0.9，Model B 是 0.89，選錯的影響極小。但如果 Model A 是 0.9，其他所有模型都只有 0.4，那這個路由決策就至關重要。
- **實作方式**：我們在損失函數中引入了 `row_weights` 與 `regret_gap_weights`。系統會計算最佳與次佳模型的 Reward 差距作為遺憾值，遺憾值越高的提問，在訓練時被賦予的權重就越高。這迫使 Router 將注意力集中在「最具鑑別度、絕對不能選錯」的高風險提問上。

## C. 事後校正與局部補償

由於 LLM 路由預測容易對特定的候選模型產生過度自信或系統性偏差，我們在 `calibrate_soft_oracle_bias.py` 實作了：

- **Bias Calibration (座標軸搜尋校正)**：驗證集的整體 Reward 是不平滑的，因此我們透過 Coarse-to-fine 的座標搜尋，針對每個候選模型的 Logit 找出最佳的平移偏差值，以最大化驗證集的 Reward，再乘上縮放因子防止過擬合。
- **KNN 殘差修正**：某些特定領域的提問可能在全局模型中表現不佳。我們利用 Embedding 模型抓出 K 個最相似的歷史提問，並計算它們的預測殘差。如果過去類似的問題某個模型的表現都被高估，就能利用 KNN 將它的預期分數扣回來，以此作為神經網路預測外的安全網。

## Q3.

在本次專案中，我們嘗試了幾種不同的路由架構與其變體，並在 Kaggle Public Leaderboard 上驗證其成效。主要的嘗試方向包含：**解耦預測與重新評分 (CARROT mean-row-max rescore)**、**軟標籤分類器 (CARROT soft-oracle classifier)** 以及**成對路由變體 (CARROT pairwise routing variant)**。

| 排名 | 方法架構 (Method)               | Public Score | 效能總結               |
|----|-----------------------------|--------------|--------------------|
| 1  | CARROT mean-row-max rescore | 0.47480      | 本次專案嘗試過的所有方法中得分最高。 |

|   |                                 |         |                                                 |
|---|---------------------------------|---------|-------------------------------------------------|
| 2 | CARROT soft-oracle classifier   | 0.46504 | 表現最好的 soft-oracle 版本，但仍低於雙軌制的解耦預測。              |
| 3 | CARROT soft-oracle classifier   | 0.46210 | 驗證集表現：routing_reward=0.460，top1_accuracy=0.224。 |
| 4 | CARROT pairwise routing variant | 0.46103 | 表現低於最佳的 soft-oracle 與解耦預測方法。                    |
| 5 | CARROT pairwise routing variant | 0.46052 | 表現與上方的成對路由變體相近。                                 |
| 6 | CARROT soft-oracle classifier   | 0.45625 | 低於表現較好的 soft-oracle 與 pairwise 版本。              |
| 7 | CARROT soft-oracle classifier   | 0.45489 | 在所有嘗試過的方法中得分最低。                                 |

表現最佳的方法：**CARROT mean-row-max rescore (解耦預測搭配成本常規化重新評分)**

此方法在 Public Leaderboard 上取得了最高分 (0.47668)，其核心設計是將「模型能力 (P)」與「運算代價 (C)」拆開來獨立預測，最後再利用作業規定的數學公式  $0.85 * P - 0.15 * C$  進行綜合評分。我們認為它表現最佳的原因如下：

1. 直接對齊最終的 **Reward** 目標函數：

不同於分類器只是預測「應該選哪個模型」，解耦預測架構直接預測了效能指標

(0.0~1.0) 與成本，並套用競賽的計算公式。這使得模型在進行路由決策時，能更精準地匹配 Kaggle 的評分機制，而非只依賴單一的機率分佈。

2. 避免了直接分類帶來的決策不穩定與分佈偏差 (Distribution Bias) :

從我們的實驗數據可以看出，**Soft-Oracle Classifier 架構** (排名 3, 4, 7, 8) 整體的表現較不穩定。在未校正的 **Soft-Oracle** 版本中，我們透過 **Validation Audit** 發現其路由分佈產生了偏差：它過度頻繁地選擇了 **Model\_H**，同時低估了 **Model\_K** 與 **Model\_C**，這與真實的最佳選擇不符。這是因為單純的分類問題很難在所有候選模型之間明確保存效能與成本的複雜權衡關係。

3. 優於成對排序 (Pairwise Routing) :

雖然 **Pairwise Routing** (排名 5, 6) 引入了成對監督機制，有助於釐清候選模型之間的相對順序，但它同樣難以精確映射到 **Reward<sub>0.85</sub>** 的絕對數值上。相比之下，**mean-row-max rescore** 不只知道「誰比誰好」，還知道「具體好多少、成本差多少」，因此在最終的混合指標上取得了最好的成效。